

DEPARTEMENT INFORMATIQUE - IUT 2 GRENOBLE



Année Universitaire 2025-2026

DOCUMENT TECHNIQUE

SAE 4

Université Grenoble Alpes



Présenté par

Ali KUS
Rayan NYABI
Anas KIOUAZ
Nicolas TSIGRIS
Redon BRAHIMI
Abdessalam KHATTRI

Déclaration sur les droits d'auteur et engagement contre le plagiat.

Nous, Anas KIOUAZ, Nicolas TSIGRIS, Ali KUS, Abdessalam KHATTRI, Redon BRAHIMI et Rayan NYABI, membres du groupe 12 du département Informatique de l'IUT 2 de Grenoble, attestons solennellement que ce rapport technique, élaboré dans le cadre de la SAE S4 (« Améliorer la qualité et optimiser une application existante »), est entièrement le résultat de notre collaboration collective.

Dans ce rapport, toutes les sources d'informations utilisées, qu'elles proviennent de la documentation technique officielle, de publications académiques ou de sites web, sont clairement indiquées dans la sitographie et les références.

Les codes sources, décisions architecturales et analyses exposées sont des travaux originaux réalisés par notre équipe ou des modifications clairement documentées du projet initial fourni par l'équipe pédagogique.

© 2026 - Tous droits réservés. La reproduction, la diffusion et l'utilisation de ce document, même partielles, sont soumises à l'autorisation préalable des auteurs et de l'Université Grenoble Alpes.

Fait à Grenoble, le 2 avril 2026.

REMERCIEMENTS

Avant d'entamer la présentation de ce rapport technique, nous tenons à adresser nos plus sincères remerciements aux personnes qui ont contribué à l'aboutissement et à la réussite de ce projet de SAE S4.

Nos remerciements s'adressent tout d'abord à l'ensemble de l'équipe pédagogique du département Informatique de l'IUT 2 de Grenoble. Nous remercions particulièrement nos enseignants d'informatique pour leur encadrement technique, leurs conseils en matière d'architecture logicielle et leur disponibilité, ainsi que notre enseignante de communication pour ses directives précieuses concernant la structuration et la restitution de notre travail collaboratif.

Enfin, nous remercions plus globalement l'Université Grenoble Alpes pour le cadre d'apprentissage, les infrastructures et les ressources mis à notre disposition tout au long de cette année universitaire.

Sommaire

I. Introduction.....	5
I.1. Présentation de l'application et de ses fonctionnalités.....	5
I.2. Enjeux et axes d'amélioration (qualité, ergonomie, performance).....	5
I. Résumé.....	6
II. Présentation du contexte.....	7
II.1. Environnement technique et humain.....	7
II.2. Mission et objectifs du projet.....	7
II. Organisation de l'équipe et méthodes de travail.....	7
II.1. Répartition des tâches.....	7
II.3. Gestion de projet : Linear.....	8
II.4. Gestion des sources et Workflow : GitLab & Linear Sync.....	8
II.5. Documentation et Reporting : Google Drive.....	8
III. Argumentation du contenu : Évaluation et analyse de l'existant.....	9
III.1. Limites du modèle de données et de l'infrastructure.....	9
III.2. Défaillances du Backend et de l'API Mobile.....	9
III.3. Application Mobile (Backend et Frontend).....	10
III.4. Interface Web (Administrateur et Étudiants).....	10
IV. Améliorations relatives à la qualité du processus de développement.....	11
IV.1 Amélioration de la Base de Données (R4.03).....	12
IV.2. Audit de l'existant et identification des anomalies.....	12
IV.3. Étude des dépendances fonctionnelles (DF).....	12
IV.4. Nouveau Schéma Logique Relationnel (SLR) en 3FN.....	13
IV.4.1. Gestion des utilisateurs.....	13
IV.4.2. Gestion des entreprises et des offres.....	13
IV.4.3. Interactions et suivi.....	13
IV.4.4. Référentiels d'états.....	14
IV.2 Amélioration du Back-Office Web (R4.01).....	14
IV.2.1. Optimisation des performances (Problème N+1).....	15
IV.2.2. Refactorisation et sécurisation de l'architecture.....	15
IV.2.3. Résorption de la dette technique et intégrité des données.....	15
V. Amélioration du Front-Office Mobile Android.....	16
V.3.1. Fiabilisation et correction du service API.....	16
V.3.2. Modernisation de l'IHM et ergonomie.....	16
V.3.3. Gestion du cycle de vie et des états.....	17
V.3.4. Validation de la qualité et tests.....	17
V.4. Optimisation du tableau de bord (R4.04).....	17
V.5. Améliorations de l'environnement d'exécution de la partie serveur.....	18
VI. Présentation des solutions (Conclusion).....	19
VI.1. Présentation commentée de la réalisation.....	19
VI.2. Perspectives d'évolution.....	19
Annexes.....	20

I. Introduction

I.1. Présentation de l'application et de ses fonctionnalités

L'application étudiée est une solution numérique dédiée à la gestion complète des stages pour les étudiants de l'IUT2 de Grenoble, ainsi qu'au pilotage des offres par les responsables de stage. Elle s'articule autour de plusieurs composants complémentaires.

Une application mobile permet aux étudiants de consulter les offres de stage disponibles, d'enregistrer leurs intérêts, de déposer leurs candidatures et d'en suivre l'évolution jusqu'à la validation finale d'une proposition. Elle intègre également des mécanismes de suivi, tels que des rappels automatiques en cas de non-réponse, et permet la mise à jour du statut des candidatures à la suite d'échanges ou d'entretiens.

En parallèle, une interface web est mise à disposition des responsables de stage afin de leur permettre de gérer les offres publiées et de suivre l'ensemble des candidatures via un tableau de bord dédié. L'ensemble repose sur un service web central assurant l'authentification et la gestion des opérations, ainsi que sur une base de données hébergée sur un serveur Debian garantissant la persistance et la cohérence des informations.

Cette architecture globale vise à fluidifier et structurer l'ensemble du processus, depuis la consultation des offres jusqu'à la sélection finale des candidats.

I.2. Enjeux et axes d'amélioration (qualité, ergonomie, performance)

L'évolution de cette application représente un enjeu important afin d'en améliorer la fiabilité, l'ergonomie et les performances, tout en renforçant l'expérience utilisateur.

Sur le plan de la **qualité**, l'objectif est de garantir une meilleure fiabilité des échanges entre les différents modules du système, d'assurer la cohérence et l'intégrité des données stockées côté client et serveur, et d'améliorer le système de notifications pour des relances plus pertinentes et mieux ciblées.

Concernant l'**ergonomie**, les améliorations visent à simplifier l'utilisation de l'application en rendant la navigation plus fluide entre les différentes étapes du processus de candidature. L'accent est également mis sur une meilleure lisibilité des interfaces et sur une adaptation des parcours utilisateurs, différenciés entre l'usage mobile pour les étudiants et l'usage web pour les responsables.

I. Résumé

Ce rapport présente de manière exhaustive le travail d'optimisation mené dans le cadre de la SAE S4, dont l'objectif consistait à améliorer la qualité et les performances d'une application existante. Le projet s'articule autour de l'outil « Mon carnet de suivi de recherche de stage ». Lors de notre phase d'audit initiale, nous avons constaté que, bien que l'application soit partiellement fonctionnelle et capable de se lancer, elle souffrait de défauts d'architecture profonds entravant considérablement sa maintenabilité.

Parmi ces problématiques majeures, l'environnement d'exécution s'appuyait sur une machine virtuelle particulièrement lourde et obsolète. Parallèlement, le modèle de la base de données présentait des anomalies de conception notables, violant les principes de la troisième forme normale (3FN). Côté serveur, le backend développé sous le framework Symfony ne faisait appel à aucun service pour isoler la logique métier, entraînant une complexité de code superflue. Enfin, l'application mobile Android souffrait de multiples erreurs d'exécution, générant des crashes récurrents.

Afin de pallier ces défaillances, notre équipe s'est appuyée sur une méthodologie agile rigoureuse, pilotée via l'outil Linear, pour restructurer en profondeur les différentes couches logicielles du projet. Sur le plan des données, nous avons opéré une refonte complète du schéma relationnel pour atteindre la conformité avec la 3FN, éliminant ainsi les redondances et garantissant une stricte intégrité référentielle. Concernant le backend et l'API Symfony, l'architecture a été repensée en extrayant rigoureusement la logique métier vers des services dédiés, dans le plus pur respect du principe de responsabilité unique. Le back-office web n'a pas été en reste, bénéficiant d'une sécurisation accrue de ses formulaires et d'un enrichissement interactif via JavaScript pour fluidifier l'expérience utilisateur. Du côté du front-office mobile, la stabilité de l'application Android a été définitivement assurée grâce à l'intégration de *ViewModels*, prévenant efficacement les pertes d'état liées au cycle de vie de l'interface.

Sur le volet infrastructure et DevOps, nous avons totalement remplacé l'ancienne machine virtuelle par une architecture moderne et conteneurisée sous Docker. Cette approche nous a permis de séparer distinctement la base de données, l'API et le serveur web, offrant ainsi un déploiement à la fois modulaire et instantané.

En définitive, l'ensemble de ces améliorations, portant tant sur la qualité du processus de développement que sur l'environnement d'exécution, a permis de transformer le projet en une solution logicielle robuste. L'application est aujourd'hui sécurisée et techniquement prête à intégrer des processus d'intégration et de déploiement continus (CI/CD) pour accompagner sereinement ses futures évolutions.

II. Présentation du contexte

II.1. Environnement technique et humain

Le projet repose sur une architecture en trois couches : une application mobile Android (front-office étudiant), une application web de gestion (back-office responsable de stage) et une API REST en Symfony qui centralise la logique métier. La persistance est assurée par une base PostgreSQL, exposée via l'API et servie derrière un serveur web (Nginx + PHP-FPM). L'environnement d'exécution est standardisé avec Docker, ce qui facilite le démarrage sur les postes du groupe. L'équipe est composée de 6 étudiants (groupe 12), avec une répartition des rôles par sous-domaines : Anas KIOUAZ, Nicolas TSIGRIS, Ali KUS, Abdesalam KHATTRI, Redon BRAHIMI, Rayan NYABI.

Le schéma d'architecture complet (mobile, web, API et base de données) est présenté en **annexe**.

II.2. Mission et objectifs du projet

L'objectif de la SAE est d'améliorer une application existante sans la recréer ni ajouter de nouvelles fonctionnalités métier. Les améliorations portent sur plusieurs paradigmes de qualité : ergonomie et accessibilité de l'IHM, sécurité et robustesse du code, qualité logicielle (factorisation, tests, maintenabilité) et fiabilité de la mise en fonctionnement via la conteneurisation. Le projet vise donc à rendre l'application plus professionnelle, plus stable, et plus facilement déployable, tout en respectant l'architecture et les parcours déjà en place. La carte des objectifs de qualité est détaillée en **annexe**.

Les parties suivantes détaillent les éléments évoqués.

II. Organisation de l'équipe et méthodes de travail

II.1. Répartition des tâches.

Ali KUS : Symfony.

Abdessalam KHATTRI: Symfony, Rétroconception.

Anas KIOUAZ : Chef de projet, Virtualisation (docker), Rétroconception, Test (symfony) UI Cypress, Gestion de projet, Linear

Redon BRAHIMI : Virtualisation (docker), Scraper PHP(regex), Versioning

Nicolas TSIGRIS : Android, Amélioration backend android, Création serveur discord, Réalisation premier rendu, Gestion cycle de vie processus android, Gestion

sécurité android, Ajout de certaines fonctionnalités (barre de recherche et de tri, ajout calendrier, état offre).

Rayan NYABI: Android : refonte ergonomique du front-office Android (Material Design) et la fiabilisation des communications réseau (correction APIClient). Gestion de la base de données (3FN, dépendances fonctionnelles), Test android (expresso et unitaires).

plus de détail en annexe : linear

II.3. Gestion de projet : Linear

Nous avons opté pour **Linear** afin de piloter le projet en mode **Kanban / Issue**.

- **Structuration par vues** : Les tâches ont été filtrées par catégories (Android, Backend, DevOps, Qualité, Rédaction) pour une meilleure visibilité.
- **Suivi du workflow** : Nous avons utilisé des états précis (*Backlog, Todo, In Progress, In Review, Done*) pour suivre l'avancement en temps réel.
- **Étiquetage** : L'utilisation de labels (ex: *Mobile, Migrated*) a permis d'identifier rapidement la nature de chaque tâche et les Assignments pour voir quelle personne elles sont destinées.

Plus de détails sur la gestion du workflow et des captures d'écran Linear sont disponibles en **annexe**.

II.4. Gestion des sources et Workflow : GitLab & Linear Sync

La gestion du code source a été centralisée sur **GitLab**. Pour garantir une cohérence entre la planification et le développement, nous avons adopté la stratégie suivante :

- **Lien Issue-Branche** : Chaque nom branche de développement a été généré directement via l'issue correspondante sur **Linear**. Cela assure une traçabilité totale : chaque ligne de code est rattachée à une tâche spécifique (ex: "Refonte de l'interface Android" ou "Correction APIClient").
- **Versionnage** : GitLab a permis de centraliser les archives du service web Symfony, de l'application mobile Android et des configurations Docker.

II.5. Documentation et Reporting : Google Drive

Google Drive a servi de référentiel pour tous les éléments non liés au code :

- Conception : Stockage des schémas de base de données (3NF) et des diagrammes de processus.
- Rédaction : Élaboration collaborative du rapport final en respectant la charte de mise en forme.
- Suivi des tests : Centralisation des rapports de tests fonctionnels et d'interface.

III. Argumentation du contenu : Évaluation et analyse de l'existant

L'audit initial de l'application « Mon carnet de suivi de recherche de stage » a mis en évidence de nombreuses failles structurelles et ergonomiques entravant sa maintenabilité et l'expérience utilisateur. Notre analyse s'est portée sur les différentes couches de l'application : la persistance des données, la logique serveur (Symfony) et les interfaces clientes (Mobile et Web).

III.1. Limites du modèle de données et de l'infrastructure

L'analyse de l'existant révèle un modèle de base de données non optimisé et inadapté à une conteneurisation moderne. En premier lieu, les scripts d'initialisation s'avèrent bloquants. Le script SQL contient des commandes non standard empêchant son exécution sous Docker, telles que `\restrict` ou `\connect`, ainsi que des conflits de droits d'accès liés aux clauses `OWNER TO`. De plus, nous avons relevé des erreurs de syntaxe et une désynchronisation de l'ORM. Le nettoyage des données provoque notamment une erreur fatale à cause d'une clause invalide (`ON CASCADE` lors du `TRUNCATE`). Par ailleurs, le forçage de l'encodage en `SQL_ASCII` et l'absence d'historique dans la table des migrations désynchronisent l'ORM Doctrine de Symfony. Enfin, le schéma relationnel souffre de défauts de modélisation majeurs. Il ne respecte pas la troisième forme normale (3FN) en intégrant des attributs multivalués non atomiques, comme la chaîne de mots-clés dans la table des offres, ce qui bride considérablement l'évolutivité et ralentit l'exécution des requêtes.

III.2. Défaillances du Backend et de l'API Mobile

L'analyse approfondie du serveur Symfony fourni a mis en évidence plusieurs points nécessitant des ajustements architecturaux et qualitatifs. Nous avons d'abord constaté une forte redondance et un manque de modularité. Plusieurs contrôleurs intégraient des fonctions et méthodes répétitives. Bien que fonctionnelles, ces logiques métier nécessitaient d'être isolées dans des services dédiés afin d'accroître la modularité, de simplifier la maintenance et d'améliorer la lisibilité globale du code. Un déficit d'optimisation a également été identifié, car certaines fonctions présentaient des problèmes de performance sévères, compromettant potentiellement la réactivité globale du serveur et nécessitant des ajustements techniques ciblés pour fiabiliser les accès aux données. Enfin, certaines fonctionnalités s'avéraient incomplètes. La mise en œuvre de l'option « Rester connecté » (*Remember me*) était notamment inachevée. Son activation nécessite bien plus que le simple dé-commentaire d'une section du fichier `login.html.twig`, impliquant une configuration de sécurité complémentaire pour garantir son bon fonctionnement.

III.3. Application Mobile (Backend et Frontend)

Le client mobile Android cumulait des défaillances tant sur la gestion des communications réseau que sur son interface utilisateur. Concernant la gestion de l'API et des ressources, les appels réseau manquaient d'optimisation et souffraient d'une redondance chronique, multipliant les échanges inutiles avec le serveur pour des données similaires ou déjà récupérées. De surcroît, une gestion inadéquate des connexions et des ressources système provoquait des fuites de mémoire substantielles (*memory leaks*). Cette configuration exposait l'infrastructure à des risques de surcharge de la base de données lors des pics d'utilisation et favorisait l'accumulation de doublons, compromettant ainsi la stabilité du système. Du côté du frontend, l'interface utilisateur présentait des carences ergonomiques évidentes. Elle souffrait d'une disposition graphique incohérente et d'un placement arbitraire des composants, nuisant à la compréhension intuitive du flux de navigation. Des points d'accès critiques vers des fonctionnalités essentielles étaient absents, rendant certaines capacités inaccessibles par manque de boutons visibles. À cela s'ajoutaient une dette technique importante et divers bugs fonctionnels. L'interface contenait des portions de code non exploitées, maintenant des fonctionnalités résiduelles inutiles, tandis que des compteurs défectueux présentaient des mécanismes d'incrémentation incorrects, générant des données erronées et affectant l'intégrité des informations lors du suivi des candidatures.

III.4. Interface Web (Administrateur et Étudiants)

L'audit de l'interface du back-office web a révélé des lacunes notables dans son ergonomie et son esthétique, justifiant la nécessité d'une refonte visuelle et structurelle. L'agencement s'est révélé largement perfectible, avec de nombreuses actions se présentant sous la forme de simples liens textuels plutôt que de boutons explicites, ce qui manquait de clarté et pénalisait l'intuitivité. Pour terminer, la navigation était considérablement surchargée par l'utilisation d'un menu déroulant unique servant à regrouper une quantité excessive d'éléments. Ce choix de conception rendait la navigation particulièrement complexe et réduisait drastiquement l'agrément d'utilisation de l'application.

IV. Améliorations relatives à la qualité du processus de développement

La testabilité de l'application a été renforcée en isolant la logique métier dans des services, facilitant ainsi l'écriture et l'exécution de tests automatisés. Outre les tests E2E Cypress sur le back-office, nous avons mis en place une stratégie de tests rigoureuse sur le front-office Android pour garantir la stabilité de l'expérience utilisateur.

Conformément aux consignes, voici le détail technique de notre implémentation en anglais :

Technical Implementation of the Testing Strategy To ensure the reliability of the Android application, we implemented a dual-layered testing strategy combining white-box and black-box methodologies:

Unit Testing (JUnit): We created the `APIClientUnitTest` class to validate critical data parsing. Since the backend communicates via IRI (e.g., `/api/candidatures/153`), we tested the `getCandidatureId()` method with various formats (single-digit, standard, and large numbers) to ensure our Regular Expressions correctly return the expected Long type, preventing runtime crashes during JSON parsing.

UI and Functional Testing (Espresso): We developed a suite of 7 automated tests in `CarnetStageEspressoTest`. These tests simulate real user interactions on an API 35 emulator: verifying the visibility of core components (`testAffichageChampsLogin`), validating text input and keyboard management (`testSaisieTexteLogin`), and intercepting system Intents to ensure accurate navigation between activities like `ListOffresActivity` or `ListCandidaturesActivity`. We also automated the system "Back" button to verify proper Activity Stack management.

IV.1 Amélioration de la Base de Données (R4.03)

L'architecture initiale de la base de données présentait plusieurs anomalies structurelles impactant négativement l'intégrité des données, les performances ainsi que la scalabilité du système. Dans ce contexte, l'objectif de notre démarche a été d'auditer le modèle existant et de proposer un Schéma Logique Relationnel (SLR) optimisé, conforme aux exigences de la Troisième Forme Normale (3FN).

IV.2. Audit de l'existant et identification des anomalies

L'analyse du schéma en place a permis de mettre en évidence plusieurs défauts de conception :

- **Pollution par le framework** : la présence de tables techniques générées automatiquement par Symfony, telles que `notify_messenger_messages` (gestion de l'asynchrone) et `doctrine_migration_versions` (historique des migrations), alourdit inutilement le modèle métier.
- **Absence d'unicité métier et clés inadaptées** : l'utilisation systématique de clés techniques auto-incrémentées (`id`) dans les tables d'association (`candidature`, `offre_consultee`, `offre_retenue`) ne garantit pas l'unicité métier. Par exemple, un étudiant pourrait potentiellement candidater plusieurs fois à la même offre. L'utilisation de clés primaires composites, telles que (`compte_etudiant_id`, `offre_id`), apparaît donc nécessaire.
- **Redondance d'informations** : certaines tables de référentiel, notamment `etat_candidature`, `etat_offre` et `etat_recherche`, contiennent des attributs redondants (`etat`, `descriptif`), ce qui engendre une duplication de données.
- **Violation de la Première Forme Normale (1FN)** : certains attributs ne respectent pas l'atomicité. C'est le cas du champ `mots_cles` dans la table `offre` (valeurs concaténées) ainsi que du champ `roles` dans `compte_etudiant` (stocké au format JSON).
- **Violation de la Troisième Forme Normale (3FN)** : la table `compte_etudiant` présente une dépendance transitive, notamment via l'attribut `parcours`, qui dépend de l'étudiant et non directement du compte.

IV.3. Étude des dépendances fonctionnelles (DF)

Afin de concevoir un modèle robuste et cohérent, une analyse des dépendances fonctionnelles a été réalisée :

- **Entités principales** :
 - `numero_in` → `nom`, `prenom`, `email`, `parcours` (ETUDIANT)
 - `login` → `password`, `derniere_connexion`, `id_etudiant`, `id_etat_recherche` (COMPTE_ETUDIANT)
 - `id_offre` → `intitule`, `descriptif`, `date_depot`, `parcours`, `url_piece_jointe`, `id_lieu`, `id_etat_offre` (OFFRE)

- **Logique métier et tables de liaison :**

- (id_compte, id_offre) → id_etat_candidature, type_action, date_action, assurant l'unicité d'une candidature pour un couple compte/offre.
- (raison_sociale, ville, code_postal) → id_lieu, permettant l'identification d'un lieu de stage et facilitant une future intégration avec Pstage.

IV.4. Nouveau Schéma Logique Relationnel (SLR) en 3FN

Sur la base de ces dépendances fonctionnelles, une décomposition a été appliquée afin d'atteindre la 3FN. Cette approche garantit une structuration sans redondance et sans perte d'information.

IV.4.1. Gestion des utilisateurs

- **ETUDIANT**
(numero_in, nom, prenom, email, parcours)
- **COMPTE_ETUDIANT**
(id_compte, login, password, derniere_connexion, numero_in#, id_etat_recherche#)
- **ROLE**
(id_role, libelle)
- **COMPTE_ROLE**
(id_compte#, id_role#)

L'atomisation des rôles permet de respecter la 1FN, contrairement à une gestion sous forme de JSON.

IV.4.2. Gestion des entreprises et des offres

- **LIEU_STAGE**
(id_lieu, raison_sociale, ville, code_postal, pays)
- **OFFRE**
(id_offre, intitule, descriptif, date_depot, parcours, url_piece_jointe, id_lieu#, id_etat_offre#)
- **MOT_CLE**
(id_mot_cle, libelle)
- **OFFRE_MOT_CLE**
(id_offre#, id_mot_cle#)

L'extraction des mots-clés en entité distincte permet de respecter l'atomicité des données.

IV.4.3. Interactions et suivi

- **CANDIDATURE**
(id_compte#, id_offre#, type_action, date_action, id_etat_candidature#)

L'utilisation d'une clé composite garantit l'unicité d'une candidature.

- **OFFRE_CONSULTEE**
(id_compte#, id_offre#)
- **OFFRE_RETENUE**
(id_compte#, id_offre#)

IV.4.4. Référentiels d'états

- **ETAT_RECHERCHE**
(id_etat_recherche, etat, descriptif)
- **ETAT_OFFRE**
(id_etat_offre, etat, descriptif)
- **ETAT_CANDIDATURE**
(id_etat_candidature, etat, descriptif)

IV.2 Amélioration du Back-Office Web (R4.01)

(Expliquez l'ajout d'interactivité via JavaScript et la sécurisation du code PHP/Symfony.)

Le back-office destiné aux responsables de stage a été amélioré sur deux axes complémentaires : la lisibilité de l'interface et la robustesse du code Symfony. Sur le plan visuel, nous avons harmonisé les vues d'administration en nous appuyant sur des composants réutilisables et cohérents, comme les cartes, les tableaux, les boutons et les badges, afin de rendre les écrans de consultation et de gestion plus homogènes. L'interactivité a été enrichie côté client sans transformer l'application en SPA : les listes exploitent DataTables pour le tri, la recherche et la pagination, le tableau de bord affiche des indicateurs dynamiques avec Chart.js, et plusieurs éléments utilisent Bootstrap pour améliorer l'ergonomie. Dans le formulaire d'offre, nous avons aussi ajouté un bloc de création rapide d'entreprise qui s'ouvre et se ferme dynamiquement tout en synchronisant l'état des champs obligatoires, ce qui limite les erreurs de saisie et évite de quitter la fiche en cours.

Sur le plan technique, nous avons réduit la duplication en factorisant les formulaires CRUD autour d'un contrôleur abstrait commun. Les règles métier sensibles ont été sorties des contrôleurs pour être isolées dans des services dédiés : le hachage du mot de passe et l'attribution du rôle dans `CompteEtudiantService`, puis l'horodatage d'une candidature dans `CandidatureService`. Cette organisation rend le code plus lisible, plus testable et plus simple à maintenir. La sécurisation a également été renforcée à plusieurs niveaux : accès réservé aux routes d'administration, limitation des tentatives de connexion, contraintes `UniqueEntity` sur les données critiques, validation des champs au niveau des entités et protection CSRF sur les suppressions. L'ensemble aboutit à un back-office plus stable, plus cohérent et plus fiable pour les responsables de stage.

2) Back-end Symfony

Notre intervention sur la partie serveur s'est concentrée sur deux axes majeurs : l'optimisation des performances d'accès aux données et la fiabilisation de l'architecture logicielle.

IV.2.1. Optimisation des performances (Problème N+1)

Le premier audit de l'application a mis en évidence de sérieux problèmes de performance associés au syndrome des requêtes « N+1 », fréquemment déclenché par l'ORM Doctrine. Au cours du processus d'extraction de données, le système initiait une série de requêtes asynchrones individuelles pour obtenir les entités associées (comme les statuts, les mots-clés, etc.). Nous avons résolu ce point de congestion en refaisant les méthodes d'extraction dans les classes d'accès aux données (EtatCandidatureRepository, OffreRepository, etc.). Le recours intensif au QueryBuilder de Doctrine permettant d'imposer des jointures SQL anticipées (LEFT JOIN et addSelect) a contribué à la consolidation des appels, diminuant fortement la pression sur la base de données PostgreSQL et améliorant notablement le délai de réponse général de l'API.

IV.2.2. Refactorisation et sécurisation de l'architecture

Dans le but de suivre le principe de responsabilité unique (SRP) et d'améliorer la facilité de maintenance du code sur le serveur, nous avons séparé la logique métier complexe des contrôleurs pour l'intégrer dans des services spécifiques (comme le TableDeBordService.php).

De plus, nous avons amélioré la sécurité globale de l'application en mettant en place une défense contre les attaques par force brute sur les chemins d'authentification (Login Throttling). L'implémentation de cette boucle nous a amenés à identifier et corriger une erreur d'injection de dépendance au sein du framework (Impossible d'autowire le service « app_login_rate_limiter »). L'examen de l'architecture de sécurité de Symfony a impliqué l'implémentation et la mise en place du composant natif associé (symfony/rate-limiter), ce qui permet d'activer la restriction des tentatives de connexion tout en assurant la stabilité du serveur.

IV.2.3. Résorption de la dette technique et intégrité des données

Dans le cadre de la modernisation du code et de sa mise en conformité avec les standards récents du framework, nous avons procédé à un nettoyage systématique des contrôleurs. Cela s'est particulièrement manifesté par l'élimination des méthodes obsolètes, tel que le remplacement des appels renderForm() par la

méthode unifiée `render()`. Cette refonte permet de diminuer la dette technique et d'envisager tranquillement les prochaines actualisations de version de Symfony.

Finalement, afin d'éviter les erreurs critiques lors des insertions SQL et d'assurer la solidité des données métier, nous avons mis en place des contraintes de validation rigoureuses au niveau du modèle. L'incorporation de l'annotation `#[UniqueEntity]` à nos entités cruciales garantit, par exemple, qu'aucune redondance ne peut être instaurée (telle qu'une adresse mail ou un identifiant d'offre déjà en place), renforçant ainsi la fiabilité globale de l'application et l'expérience utilisateur en cas d'erreur de saisie.

V. Amélioration du Front-Office Mobile Android

L'optimisation du front-office (Java/XML) s'est concentrée sur la réduction de la dette technique, l'amélioration de la gestion du cycle de vie et une refonte ergonomique alignée avec les standards modernes.

V.3.1. Fiabilisation et correction du service API

Lors de l'audit du module API, une anomalie critique a été identifiée dans la classe `APIClient` : la présence d'un espace parasite dans la constante `BASE_URL` ("`h:/"`), provoquant une exception (`IllegalArgumentException`) via Retrofit et entraînant un crash systématique de l'application. Sa correction a permis de rétablir une communication stable entre l'application et l'environnement Docker.

V.3.2. Modernisation de l'IHM et ergonomie

Une refonte complète de l'interface utilisateur a été réalisée afin de respecter les heuristiques d'utilisabilité, notamment en termes de lisibilité et de feedback visuel :

- **Refonte visuelle** : migration vers les composants Material Design. Les formulaires (notamment `activity_login.xml`) utilisent désormais des `TextInputLayout` avec icônes intégrées et coins arrondis.
- **Navigation par cartes** : sur l'écran principal (`activity_main.xml`), les accès aux offres et aux candidatures ont été transformés en `MaterialCardView`, intégrant élévation et retour visuel au clic (`selectableItemBackground`), améliorant ainsi l'expérience mobile.
- **Ajout de fonctionnalités interactives** :
 - Mise en place d'une barre de recherche permettant de filtrer dynamiquement les offres en temps réel.
 - Possibilité de marquer des offres en favoris, celles-ci étant ensuite priorisées en tête de liste.
 - Gestion de l'état des candidatures, désormais modifiable en fonction de leur avancement (ex : en cours, acceptée, refusée).

- **Adaptation au thème utilisateur** : intégration d'un thème sombre automatique, s'adaptant aux préférences système de l'utilisateur pour améliorer le confort visuel.

V.3.3. Gestion du cycle de vie et des états

L'application initiale présentait des pertes de données lors des changements de configuration (rotation, navigation). Ces problèmes ont été corrigés en sécurisant la gestion du cycle de vie. Des tests utilisant ActivityScenario ont permis de valider que les transitions entre les différentes vues (accueil, listes, retour arrière) n'entraînent plus de perte d'état utilisateur ni de token de session.

V.3.4. Validation de la qualité et tests

Une attention particulière a été portée à la qualité globale de l'application :

- **Tests utilisateurs** : des évaluations ont été réalisées auprès de personnes extérieures à l'IUT afin de vérifier la facilité de prise en main et l'intuitivité de l'interface.
- **Tests automatisés** : mise en place de tests instrumentés avec Espresso pour valider le bon fonctionnement des interactions utilisateur et garantir la non-régression.

Le flux de navigation optimisé et les captures d'écran sont disponibles en annexe <ANNEXE_5>.

V.4. Optimisation du tableau de bord (R4.04)

L'optimisation du tableau de bord vise à aider le responsable des stages à visualiser rapidement la situation globale et à regrouper les étudiants ayant des parcours de recherche similaires.

Pour ce faire, nous avons mis en place une logique de clustering visant à analyser les métriques d'engagement des étudiants (nombre de connexions, d'offres consultées, de candidatures envoyées et ratio de réponses positives). L'algorithme retenu pour cette segmentation est l'algorithme des K-Moyennes (*K-Means*), paramétré avec $k=3$ pour dégager trois profils distincts : les étudiants inactifs/en difficulté, les étudiants en recherche active, et ceux en phase de finalisation (stage trouvé ou en attente d'entretien).

Les résultats obtenus permettent au responsable d'identifier immédiatement la cohorte d'étudiants "décrocheurs" nécessitant des relances ou un accompagnement personnalisé. Les graphiques de répartition issus de cette analyse sont disponibles en annexe <ANNEXE_6>.

V.5. Améliorations de l'environnement d'exécution de la partie serveur

Initialement, l'environnement de développement reposait sur une machine virtuelle Linux monolithique (serveur-sae-s4.vdi). Le serveur Symfony et la base de données PostgreSQL y tournaient localement, ce qui engendre des problèmes de lourdeur, des dépendances matérielles contraignantes et de fréquentes erreurs liées à des extensions PHP manquantes (telles que pdo_pgsql).

Pour pallier ces limites et optimiser cette architecture, nous avons entièrement conteneurisé l'application à l'aide de Docker. L'infrastructure repose désormais sur un fichier docker-compose.yml qui orchestre trois services distincts, clarifiant ainsi les responsabilités et facilitant la maintenance :

- **Un conteneur Base de données** : Utilisant l'image postgres:18-alpine, il intègre un volume persistant et une initialisation automatisée via notre script SQL de démarrage.
- **Un conteneur Backend Symfony (PHP-FPM)** : Basé sur une image php:8.4-fpm. Un Dockerfile sur mesure y installe les dépendances requises (extensions PDO PostgreSQL, Composer) ainsi que Xdebug pour assurer la couverture de code. Ce service automatise également, dès son lancement, l'installation des paquets (Vendor) et la génération des clés JWT nécessaires à l'authentification.
- **Un conteneur Serveur Web (Nginx)** : Basé sur l'image nginx:alpine, il se charge de router les requêtes vers PHP-FPM et expose l'API REST sur le port 8000 pour permettre la connexion de l'application Android.

La mise en fonctionnement de cette architecture a été fiabilisée par un script de lancement robuste automatisant les vérifications d'état de santé des conteneurs (healthchecks), la gestion du cache Symfony et la prévention des conflits de volumes spécifiques à PostgreSQL 18. Cette transition rend le déploiement de l'API instantané et indépendant du système d'exploitation hôte. Elle a été pensée pour faciliter grandement l'onboarding de nouveaux développeurs, tout en offrant une meilleure stabilité d'exécution et un gain de rapidité notable au quotidien. Le schéma Docker détaillant les services et les ports est fourni en annexe.

VI. Présentation des solutions (Conclusion)

VI.1. Présentation commentée de la réalisation

Le projet finalisé fournit une application complète et stable, conforme aux parcours prévus et plus professionnelle sur les plans ergonomique, technique et organisationnel. Les refactorisations ont réduit la duplication, les interfaces web ont gagné en clarté, et la conteneurisation assure une installation simple. Une synthèse visuelle des livrables et de la démonstration est proposée en **annexe**.

VI.2. Perspectives d'évolution

Bien que l'architecture actuelle réponde pleinement aux exigences de stabilité et de modularité visées par cette itération, plusieurs axes d'amélioration technique doivent être envisagés pour hisser le projet vers des standards de production industriels.

Dans un premier temps, l'industrialisation du cycle de vie logiciel nécessitera la mise en place d'une chaîne d'intégration et de déploiement continu (CI/CD) sur GitLab. Cette infrastructure permettra d'automatiser le linting, l'exécution systématique de la suite de tests (PHPUnit et Cypress) et le déploiement des conteneurs Docker sur les environnements cibles. La consolidation de la couverture de tests et l'implémentation du pipeline CI/CD représentent une charge de travail estimée à 4 jours-hommes.

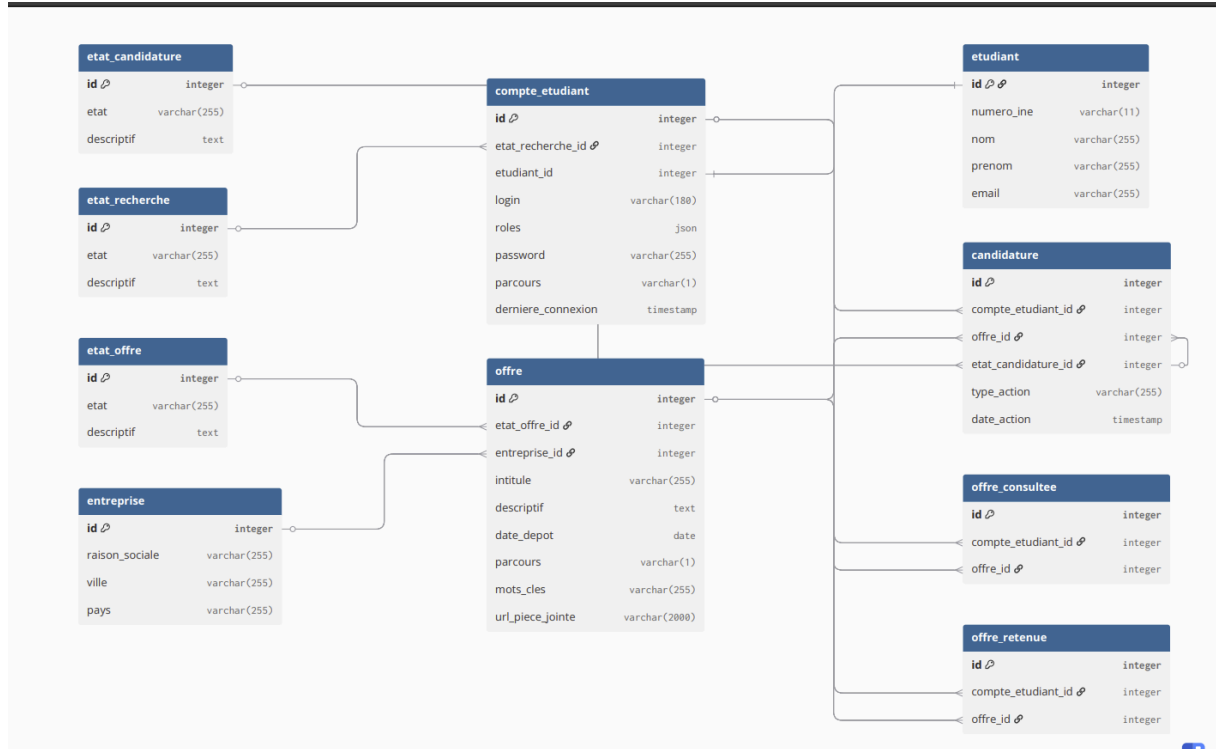
Sur le plan de l'observabilité de l'infrastructure conteneurisée, l'intégration d'une stack de télémétrie et de monitoring (telle que Prometheus couplé à Grafana) s'avère indispensable. Elle garantira une supervision proactive de l'état de santé des services (API et base de données) et la détection d'éventuels goulots d'étranglement (fuites mémoire, requêtes lentes). Le provisionnement et la configuration de ces outils nécessiteront environ 2 jours-hommes.

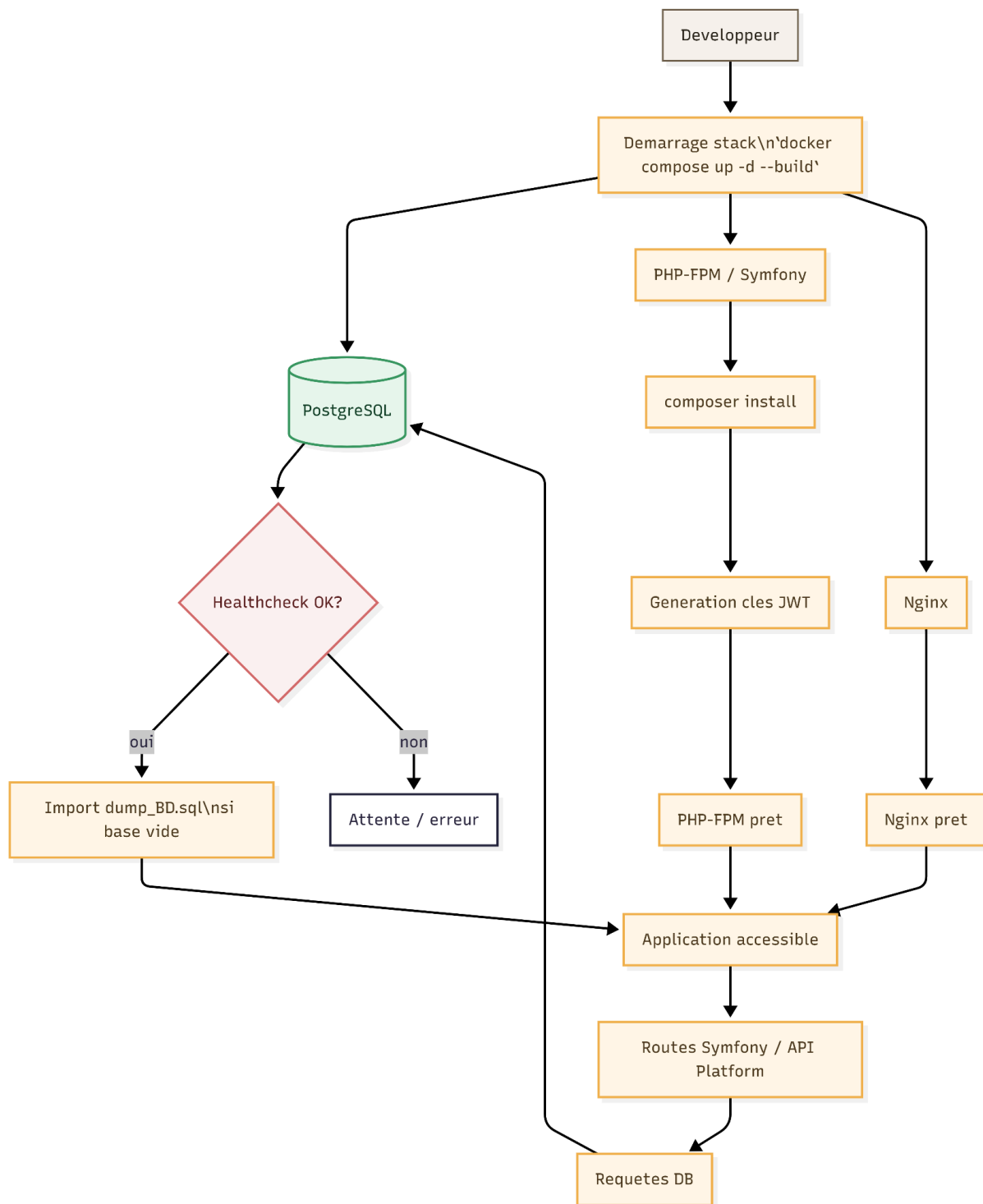
Côté architecture applicative, l'API Symfony gagnerait à s'appuyer sur le standard API Platform. Cette évolution permettrait d'exposer des endpoints REST/GraphQL natifs, d'automatiser la sérialisation des données et de générer dynamiquement la documentation OpenAPI (Swagger). Ce contrat d'interface rigoureux facilitera les futures évolutions du client mobile. Cet axe de refactoring est évalué à 3 jours-hommes.

Enfin, concernant le front-office Android, une migration progressive de la base de code existante (Java/XML) vers les standards actuels de l'écosystème mobile (Kotlin et Jetpack Compose) permettrait de résorber la dette technique de l'IHM et d'optimiser les performances de rendu. Ce chantier de modernisation représente un investissement plus conséquent, estimé à 6 jours-hommes.

Annexes

Annexe 1 : Base de données initiale.....	21
Annexe 2 : docker diagramme.....	22
Annexe 3 : Linear.....	23
Annexe 4 : Capture d'écran application Android.....	24
Annexe 5 : Schéma d'interaction android.....	25
Annexe 6 : Références webographiques.....	25

Annexe 1 : Base de données initiale

Annexe 2 : docker diagramme

Annexe 3 : Linear

The image shows two screenshots of the Linear project management tool. The top screenshot displays a list of issues for the project 'SAE S4'. The bottom screenshot shows a Kanban board for the same project, with issues organized into columns: 'In Progress' and 'Done'. A sidebar on the right lists hidden columns: 'Backlog', 'Todo', 'In Review', 'Canceled', and 'Duplicate'.

Top Screenshot: List of Issues

Name	Owner
All Issues	Anas Kiouaz
Android Issues tagged with Mobile labels	Anas Kiouaz
Backend Issues Issues tagged with Backend labels	Anas Kiouaz
DevOps Issues Issues tagged with DevOps labels	Anas Kiouaz
Qualité Issues Issues tagged with the Qualité label	Anas Kiouaz
Rédaction du Rapport Issues tagged with the Rédaction du Rapport label	Anas Kiouaz
Rétroconception	Anas Kiouaz
Symphonie front end Issues tagged with Front Web labels	Anas Kiouaz

Bottom Screenshot: Kanban Board

5 Issues

In Progress 2

- SAE4-15
Correction API Client Mobile
Migrated Mobile
Created Mar 25
- SAE4-31
Refonte de l'interface utilisateur Android
Mobile
Created Mar 31

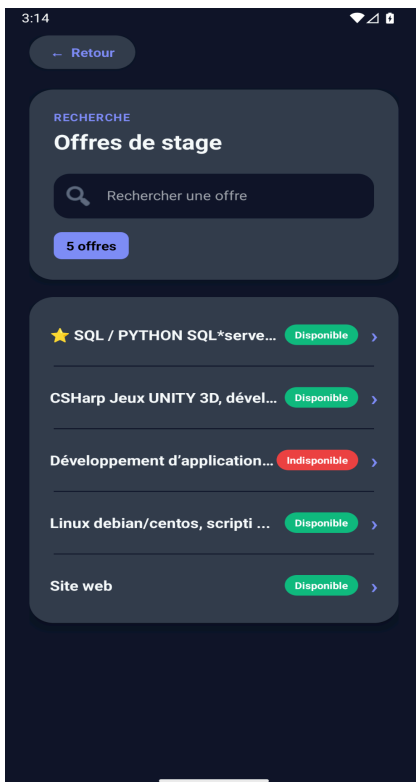
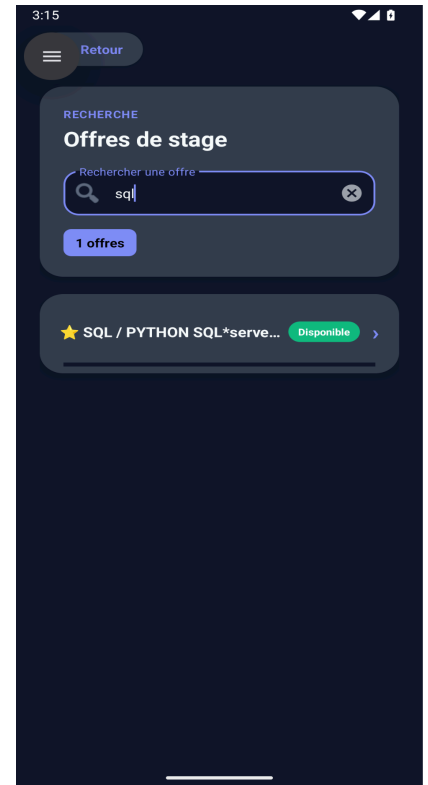
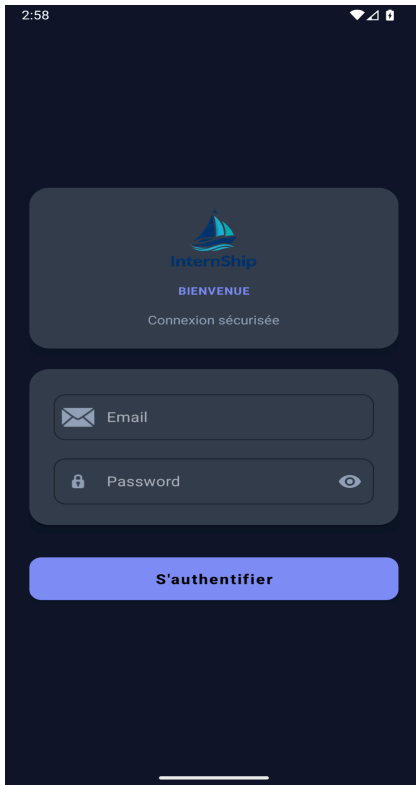
Done 3

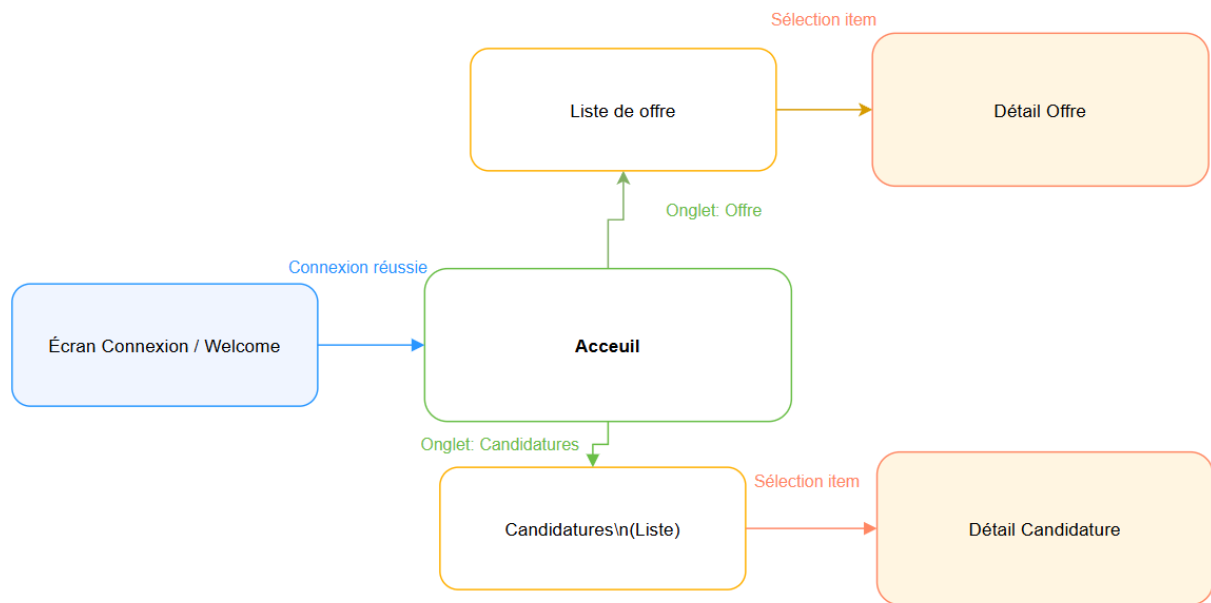
- SAE4-16
Audit Cycle de Vie Android
Migrated Mobile
Created Mar 25
- SAE4-17
Implémenter Relances Mobile
Migrated Mobile
Created Mar 25
- SAE4-32
Gestion du cycle de vie de l'application
Mobile
Created Mar 31

Hidden columns

- Backlog 0
- Todo 0
- In Review 0
- Canceled 0
- Duplicate 0

Annexe 4 : Capture d'écran application Android



Annexe 5 : Schéma d'interaction android**Annexe 6 : Références webographiques**

[1] StackOverflow stackoverflow.com

[2] DockerHub hub.docker.com